Rowan University
Department of Computer Science

Performance Profiling for C++ Serialization Libraries

Aaron Boylan, Jacob Flores, Colin Harrell, Kareena Khan, Evan Marino

Project Introduction

- **What is Serialization?**
  - It is the process of converting data in code into a stream of bytes to save the state of an object in easily transmittable code
- **Objectives:** Identify data serialization options for C++ code in a Linux Environment
  - Understand and compare different serialization methods
  - Generate comprehensive report on results
    - Comparing all the Libraries against each other
    - Recommend a library based on the finding
- **Environment:** Ubuntu Linux operating on Oracle VirtualBox

Test Bench Introduction

- The Test Bench was created using CMake to build and link all of the necessary libraries and header files.
- It creates an instance of each Serialization library
  - Boost.Serialization
  - Cereal
  - Flatbuff
  - Protobuf
- It creates a Test Object consisting of:
  - 1 Array of 5 Integers
  - 3 Integers
  - 3 Floats
  - 4 C++ standard strings
- It then Serializes and Deserializes this Test Object using each library 'X' times. 'X' is set by the user in the command line.
- It then tracks the average time for these functions to take place, each on their own thread
- Then finds the size of the final serialized file
- Finally, it writes the output to the terminal

```
public:
    TestObject(){};

    TestObject(
        std::array<int, 5> integers,
        std::string string1,
        std::string string2,
        std::string string3,
        std::string string4,
        int int1,
        int int2,
        int int3,
        float float1,
        float float2,
        float float3
    ){
```

```
Number of Iterations: 1000

Original Test Object:
        8823, 2281, 9224, 6554, 9417
        'string1', 'string2', 'string3', 'string4'
        597, 4340, 4337
        7325, 3281.7, 1496.23

Boost:
        Object is correctly deserialized
        Average Time: 1063 microseconds
        boostTestObject.bin is 166 bytes

Cereal Binary:
        Object is correctly deserialized
        Average time: 1036 microseconds
        cerealTestObject.bin is 104 bytes

Cereal XML:
        Object is correctly deserialized
        Average time: 1026 microseconds
        cerealTestObject.xml is 479 bytes

Cereal JSON:
        Object is correctly deserialized
        Average Time: 1108 microseconds
        cerealTestObject.json is 401 bytes

Flatbuff:
        Object is correctly deserialized
        Average Time: 914 microseconds
        flatbuffTestObject.bin is 144 bytes

Protobuff:
        Object is correctly deserialized
        Average Time: 899 microseconds
        protobuffTestObject.bin is 72 bytes
```

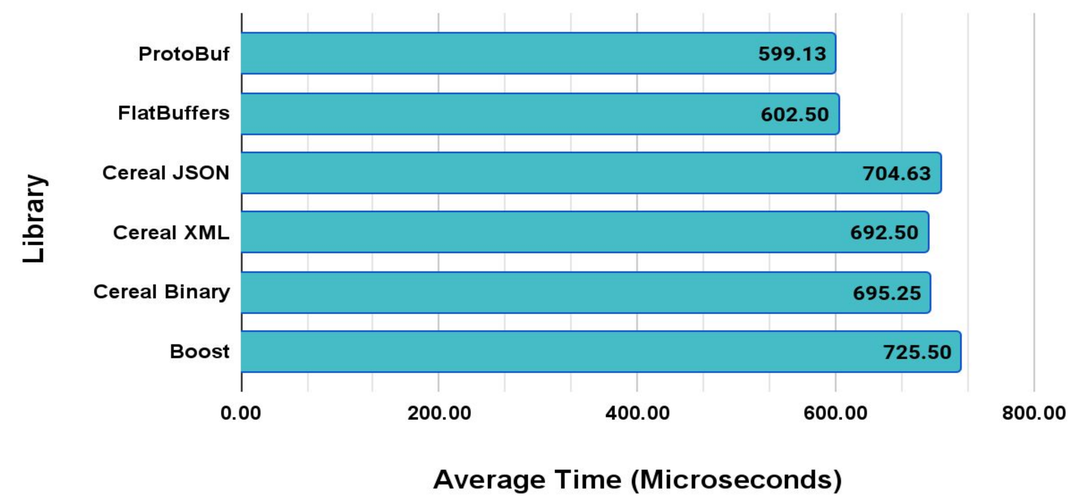
Observational and Researched Qualities

- **Ease of Use:**
  - Cereal was the easiest because it was header only, so it was an easy download and implementation process
  - ProtoBuf was the hardest due to compatibility issues with the latest versions
- **Community Support**
  - All Libraries had some sort of forum, typically a Google Group or/and a StackOverflow to ask questions, but most were unreliable in either form
  - The worst was Boost with only a Mailing List that had to be subscribed to
  - The best was Flatbuffers which had a Discord, Google Group, StackOverFlow, Gitter Chat Room, and Resources on their Overview Page
- **Security History**
  - Security Vulnerabilities were checked with CVE Details who uses two scoring systems, EPSS and CVSS:
  - No versions of the Libraries that were used were found with vulnerabilities, just previous version had minor issues

Numerical Qualities

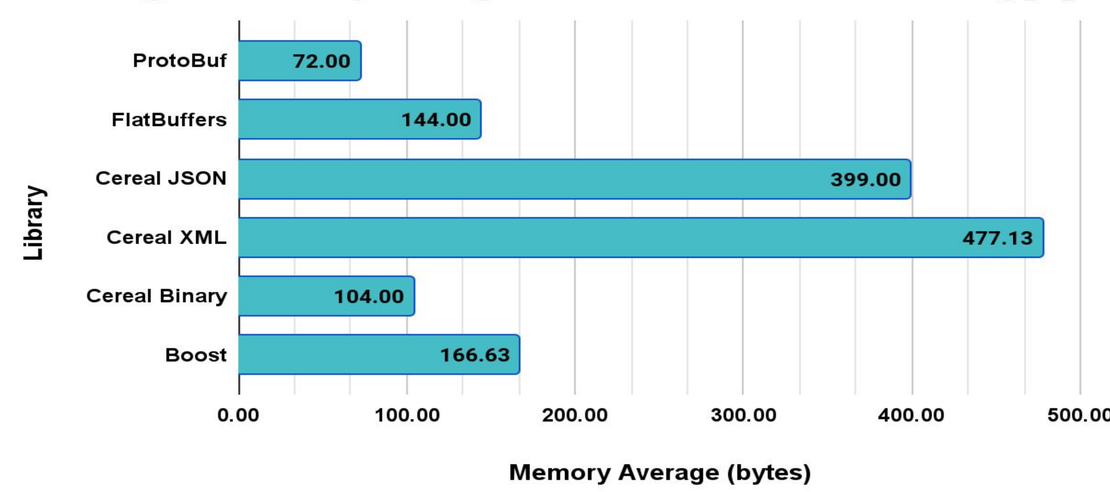
- **Speed:**
  - Measured in microseconds to see how long it takes to serialize and deserialize the input data
  - Tested with different instances of runs (10-1,000,000 runs) (Fig[2]) to get an average speed for each Library

Average Speed Across all Runs **Fig[2]**



- **Memory Usage:**
  - Measured in Bytes
  - Tested with different instances of runs (10-1,000,000 runs) ([Fig4])s

Average Memory Usage Across All Runs **Fig[4]**



- **Data Integrity:**
  - Tested with different instances of runs (10-1,000,000 runs)
  - Each pre-serialized and post-serialized file was test to see if their contents matched after the serialization process
  - Each Library was found to be equally reliable and was able to maintain the contents of the object it was serializing

Is the Object Correctly Deserialized?								
Library	10 runs	100 runs	500 runs	1,000 runs	10,000 runs	100,000 runs	500,000 runs	1,000,000 runs
ProtoBuf	yes	yes	yes	yes	yes	yes	yes	yes
FlatBuffers	yes	yes	yes	yes	yes	yes	yes	yes
Cereal JSON	yes	yes	yes	yes	yes	yes	yes	yes
Cereal XML	yes	yes	yes	yes	yes	yes	yes	yes
Cereal Binary	yes	yes	yes	yes	yes	yes	yes	yes
Boost	yes	yes	yes	yes	yes	yes	yes	yes

Overall Recommendation & Conclusion

- ASRC currently uses raw serialization to transfer data between devices, but we concluded that the best option comes down to whether Ease of Use or Performance is more important as
  - ProtoBuf has the best performance numbers overall
  - Cereal is by far the easiest to use
  - FlatBuffers has the best Community Support and is in the Top 3 for both Speed and Memory